

操作系统

第3章 外设与文件

(2) 文件管理（上 文件系统静态结构）

同济大学计算机系



一、基本概念（磁盘上的文件和文件系统）

- 文件系统存放很多文件。
- 文件，是可重复访问的字节序列。支持随机访问、所有字节可重复读写。
- 用户文件，是文件系统的用户数据。
 - 可执行文件供进程管理系统使用
 - 普通数据文件供应用程序使用。
- 元数据（metadata），是文件系统管理用户文件所需的磁盘上数据结构。
 - 每个文件一个文件控制块，登记文件访问权限、文件数据块存放的位置。读写文件，需要使用文件控制块。
 - 文件树，挂所有文件。目录搜索，就是在文件树上找文件。
- 用户数据和元数据存放在同一张磁盘。

Unix和Windows，完全不同的文件系统风格
首先介绍Unix。

二、文件、逻辑块、物理块

- 文件按磁盘数据块大小切块，离散存放在磁盘上。每一块是一个逻辑块。
- 为文件分配的磁盘存储空间，是整数个磁盘数据块。
- 文件系统使用离散存储管理方式。
 - 相邻的文件逻辑块在磁盘上不必连续存放。但尽量连续存放一定能提升文件顺序读写效率。

例：Unix V6++，磁盘数据块512字节。

- 一个size 是 1000 字节的文件有2个逻辑块，0~511字节是0#逻辑块，512~999#字节是1#逻辑块。系统需要为该文件分配1024字节磁盘空间。这2个磁盘数据块不必相邻。

磁盘数据块简称物理块



三、文件控制块 (File Control Block, 简称FCB)

每个文件有一个文件控制块

- 文件名, 文件尺寸
- 地址映射表, 用来计算分配给逻辑块bn的物理块号dn。使用方法:
 - $dn = f(bn)$; /* $bn = \text{offset}/512$; offset是数据在文件中的偏移量 */
 - 数据存放在物理块dn, 块内偏移量是 $\text{offset}\%512$
- 文件主 uid, 创建该文件的进程的uid
- 文件访问权限, 登记各类用户对该文件的读、写、执行权限
 - Unix 是 **RWX****RWX****RWX**, 标识**文件主**, **文件所在的用户组**和nobody对文件的读、写、执行 权限
 - Windows 是 ACL (Access Control List)
 - 进程 uid 和 gid 标识使用文件的用户 以及 用户所在的组
- 创建时刻, 最后一次访问时刻, 最后一次修改时刻。



FCB的创建

- 创建文件时，进程执行系统调用 `create(文件名, mode)`。mode 是文件的访问权限 `rw-rw-rw-`
 - 新建FCB
 - 初始化FCB：填入文件名，uid=进程uid，访问权限=mode。文件尺寸=0，创建时刻=time

FCB的使用

- 访问已有文件，进程需要使用FCB。
 - 使用文件前，执行open系统调用打开文件。
 - `open ( “文件名” , mode)`。mode 是进程希望对文件执行的操作：读、写 还是 既读又写。
 - 需要将磁盘上的FCB读入内存，使用访问权限字段，匹配FCB中的mode，判断进程（p_uid用户）有没有文件的访问权限。
 - 读写文件时，进程执行read/write系统调用。
 - 需要使用FCB中的地址映射表确定需要读写的磁盘数据块
 - 执行bread/bwrite/breada.....合适的函数，进行字符块读写
 - 文件访问结束，执行close系统调用关闭文件。
 - 写文件会导致文件长度和地址映射表发生变化
 - 内存中的FCB，如果发生变化，写回磁盘。

Windows的文件控制块 和 Unix的文件控制块

- Windows 系统中，文件控制块 是 目录项。这个数据结构存放在目录文件里。
- Unix 系统，文件控制块存放在 2 处：
 - 文件名存放在目录项 (DirectoryEntry)
 - 其余属性存放在索引节点 (inode, i 是index 索引的缩写)。

目录项

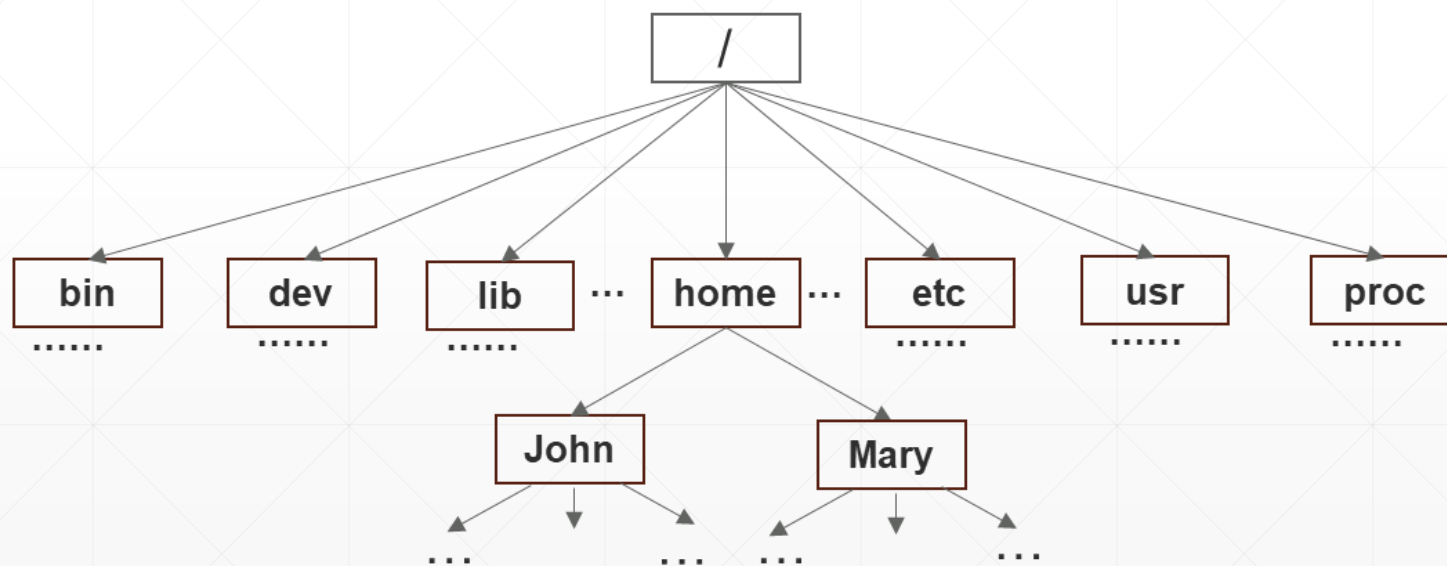
文件名	文件ID(inode号)
-----	--------------

其余属性

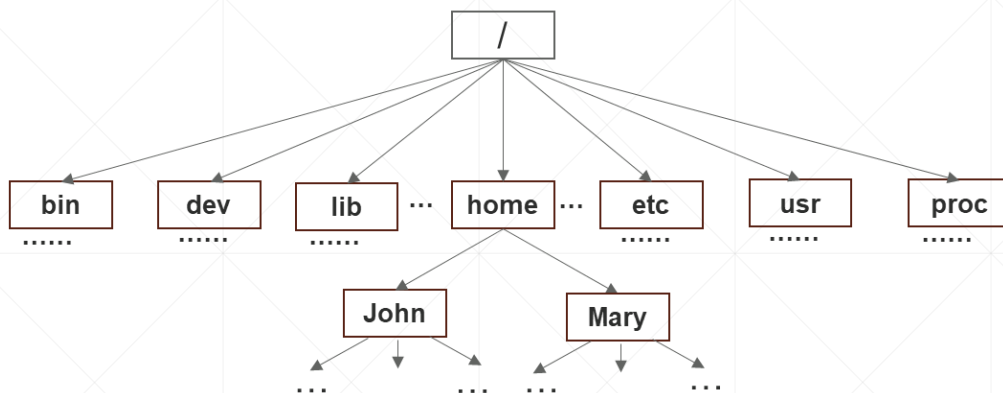
inode

四、文件树，目录项 和 目录文件

- 文件树，节点是文件，边是目录项。
 - 叶子节点是数据文件。
 - 非叶子节点是目录文件，该节点的每条出边是一个目录项，对应该目录下的一个文件或一个子目录。



Unix的目录文件



目录文件是等长记录文件。
每条记录**32**字节，是一个目录项，表示文件树上的一条边。

```

class DirectoryEntry
{
    int m_ino; // 边的终点，文件ID
    char m_name[DIRSIZ]; // 文件名
};

```

.	本目录
..	父目录
ID	文件名

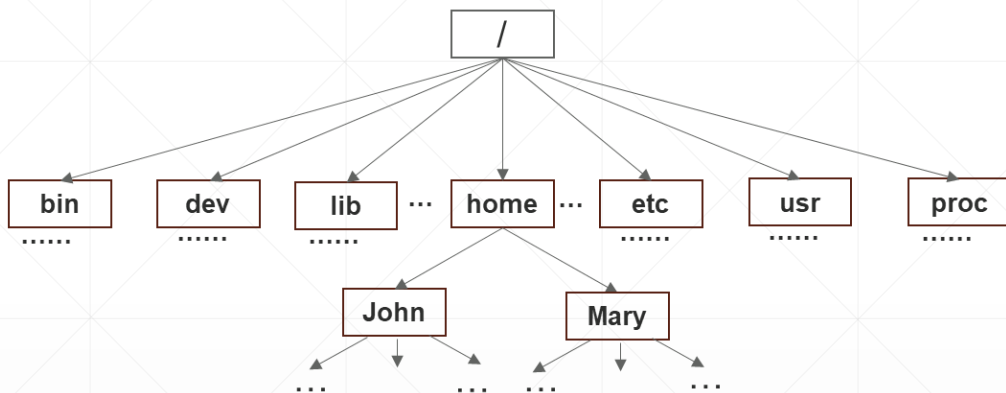
特殊路径名（文件名）

(1) . 出现在命令行 或 **PATH** 环境变量，表示当前工作目录。

出现在目录文件，表示本目录。

(2) .. 用于目录搜索，表示当前目录的父目录。这个特殊目录项让目录搜索可以沿文件树向上走，用来支持相对路径。

Unix的目录文件 (示例)



文件ID是正整数。
1是根目录。
0不用，表示空目录项。

根目录文件

1	.
1	..
2	bin
3	dev
4	lib
5	home
6	etc
7	usr
8	proc

home目录文件

5	.
1	..
26	John
0	已删除目录项
32	Mary
0	null
0	null
0	null
.....	

登记所有用户
的家目录

空目录项

绝对路径名、相对路径名 和 目录搜索

文件系统按名查找文件（使用文件路径名搜索文件树）。路径名是文件的外部引用名，有2种：

- 绝对路径名，从根目录节点到该节点的路径。

文件b的绝对路径名： `/home/John/b`

- 文件的相对路径名，从进程当前工作目录到该节点的路径。

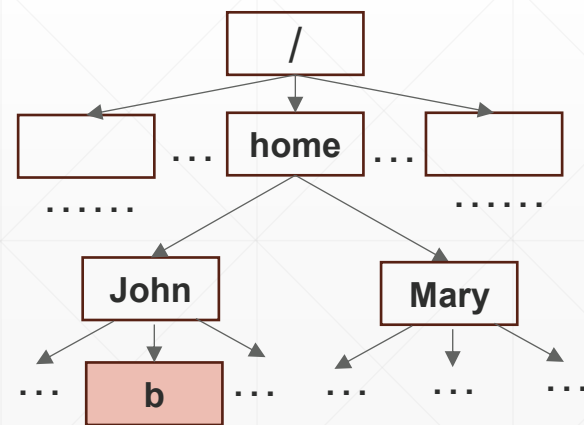
- John用户成功登录后，当前工作目录 `u_cdir` $\rightarrow$ `/home/John`，文件b的相对路径名： `b`

- 支持相对路径，有利于缩短路径名的长度，降低查找文件的开销。

- 支持相对路径名的机制： `u_cdir` 和 `..`。.. 所有节点可达。

- 本例：Mary用户，引用文件b，使用的相对路径名： `../John/b`

- 执行 `cd` 命令，shell进程改变 `u_cdir`，并影响随后创建的所有子进程的当前工作目录。



五、Disklnode 每个文件（包括目录文件），磁盘上有一个Disklnode

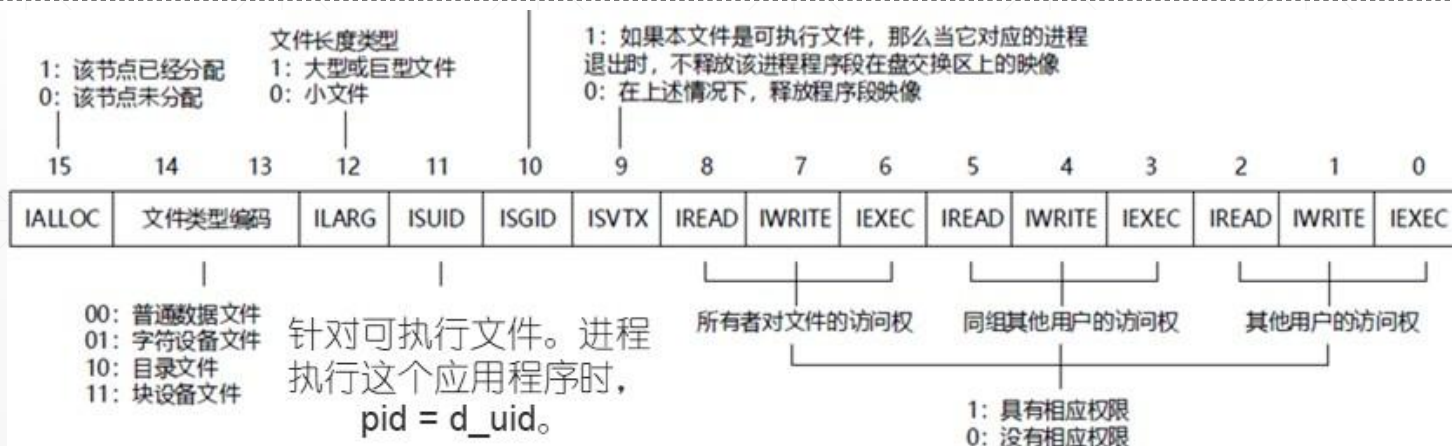
Disklnode是Unix的文件控制块。Disklnode号是文件ID，文件的内部标识

```
class Disklnode
```

```
{
unsigned int d_mode;
int          d_nlink;    /* 硬链接数 */
short        d_uid;      /* 文件主的uid */
short        d_gid;      /* 文件所在的组 */
int          d_size;     /* 文件尺寸，字节数 */
int          d_addr[10]; /* 地址映射表 */
int          d_atime;     /* 最后访问时刻 */
int          d_mtime;     /* 最后修改时刻 */
}; // 64字节
```

```
IALLOC = 0x8000; /* 文件被使用 */
IFMT = 0x6000; /* 文件类型掩码 */
IFDIR = 0x4000; /* 文件类型：目录文件 */
IFCHR = 0x2000; /* 字符设备特殊类型文件 */
IFBLK = 0x6000; /* 块设备特殊类型文件，为0表示常规数据文件 */
ILARG = 0x1000; /* 文件长度类型：大型或巨型文件 */
ISUID = 0x800; /* 执行时文件时将用户的有效用户ID修改为文件所有者的User ID */
ISGID = 0x400; /* 执行时文件时将用户的有效组ID修改为文件所有者的Group ID */
ISVTX = 0x200; /* 使用后仍然位于交换区上的正文段 */
IREAD = 0x100; /* 对文件的读权限 */
IWRITE = 0x80; /* 对文件的写权限 */
IEXEC = 0x40; /* 对文件的执行权限 */
IRWXU = (IREAD | IWRITE | IEXEC); /* 文件主对文件的读、写、执行权限 */
IRWXG = ((IRWXU) >> 3); /* 文件主同组用户对文件的读、写、执行权限 */
IRWXO = ((IRWXU) >> 6); /* 其他用户对文件的读、写、执行权限 */
```

d_mode, Disklnode的使用状态
文件类型 和 访问权限



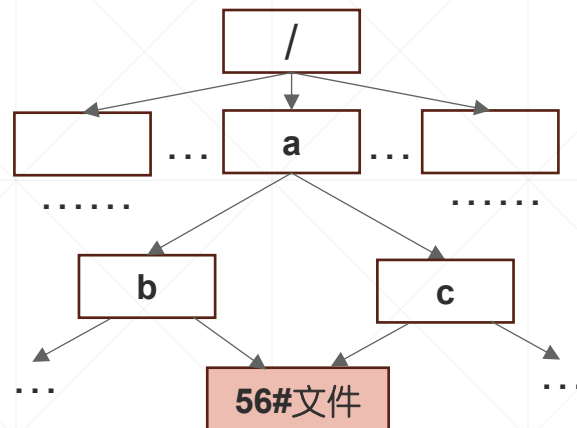
文件的硬链接

56# DiskNode

d_nlink==2

2个目录项引用56#文件

"filename1\0"	56
"filename2\0"	56



引用56#文件的2个文件名:
/a/b/filename1
/a/c/filename2

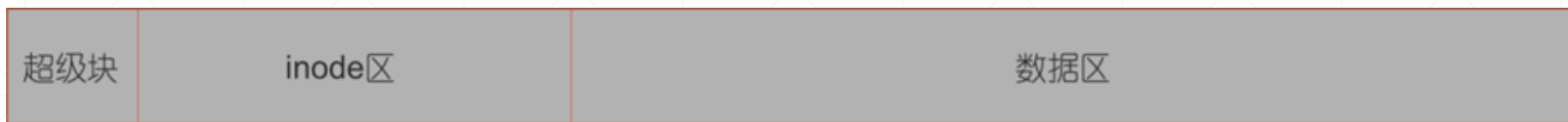
文件的软链接 (windows叫做快捷方式)

软链接，是一个小文本文件，存放目标文件的绝对路径名。有目录项，有DiskNode，有数据块。
打开软链接：系统需要读出绝对路径名，open这个文件。

六、文件卷

- 文件卷是持久化存储文件系统的磁盘。1个文件系统，1个卷，存放其管理的用户文件和运行所需的元数据。
- 卷格式取决于具体的文件系统。Unix文件卷 和 Windows文件卷格式完全不同。

Unix V6文件卷分 3 个区域：超级块、inode区 和 数据区

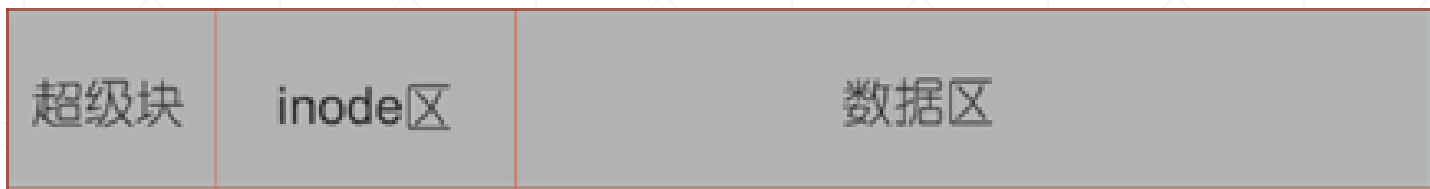


- 超级块，存放文件卷配置信息 和 文件系统的使用状态。0#、1#扇区
- inode区存放文件控制块DiskInode，从2#扇区开始
- 数据区存放普通数据文件和目录文件的数据块和索引块

超级块

```
11 class SuperBlock
12 {
13
21 public:
22     int     s_isize;    /* 外存Inode区占用的盘块数 */
23     int     s_fsize;    /* 盘块总数 */
24
25     int     s_nfree;    /* 直接管理的空闲盘块数量 */
26     int     s_free[100]; /* 直接管理的空闲盘块索引表 */
27
28     int     s_ninode;    /* 直接管理的空闲外存Inode数量 */
29     int     s_inode[100]; /* 直接管理的空闲外存Inode索引表 */
30
31     int     s_flock;    /* 封锁空闲盘块索引表标志 */
32     int     s_iloc;    /* 封锁空闲Inode表标志 */
33
34     int     s_fmod;    /* 内存中super block副本被修改标志，意味着需要更新外存对应的Super Block */
35     int     s_ronly;    /* 本文件系统只能读出 */
36     int     s_time;    /* 最近一次更新时间 */
37     int     padding[47]; /* 填充使SuperBlock块大小等于1024字节，占据2个扇区 */
38 };
```

磁盘资源 和 空闲资源管理



数据区的分配单位是物理块
Unix V6, 1个扇区



inode区的分配单位是DiskInode (64字节)

DiskInode的编号, 由存储位置决定。 n # DiskInode存放位置: 扇区号 $n/8 + 2$, 扇区内偏移量是 $(n\%8) * 64$



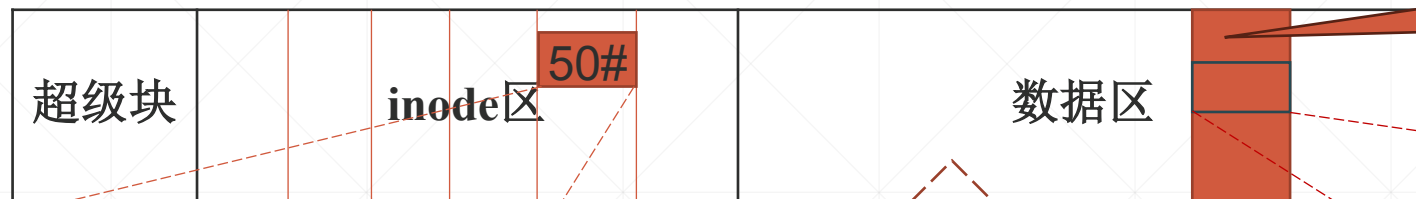
七、文件长啥样

- 磁盘上的普通数据文件
- 磁盘上的目录文件
- 特殊文件（外设硬件）

1 磁盘上的普通数据文件

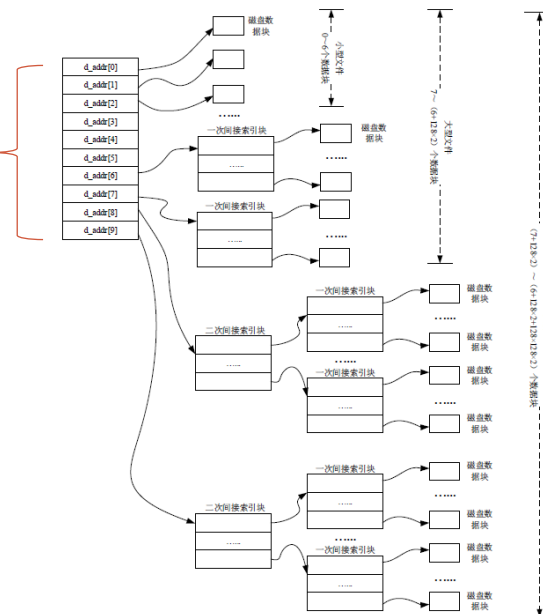
/home/John/b

1个DiskNode, 1个目录项, 1棵混合索引树 (树上挂着分配给它的数据块和索引块)



父目录John的
某个扇区

```
class DiskNode
{
public:
    文件类型: 00
    unsigned int d_mode;
    int d_nlink;
    short d_uid;
    short d_gid;
    int d_size;
    int d_addr[10];
    int d_atime;
    int d_mtime;
};
```



```
class DirectoryEntry
{
public:
    static const int DIRSIZ = 28;
public:
    int m_ino; // 目录项中Inode编号
    char m_name[DIRSIZ]; // 目录项中路径名部分
};
```

50	"b\0"
----	-------

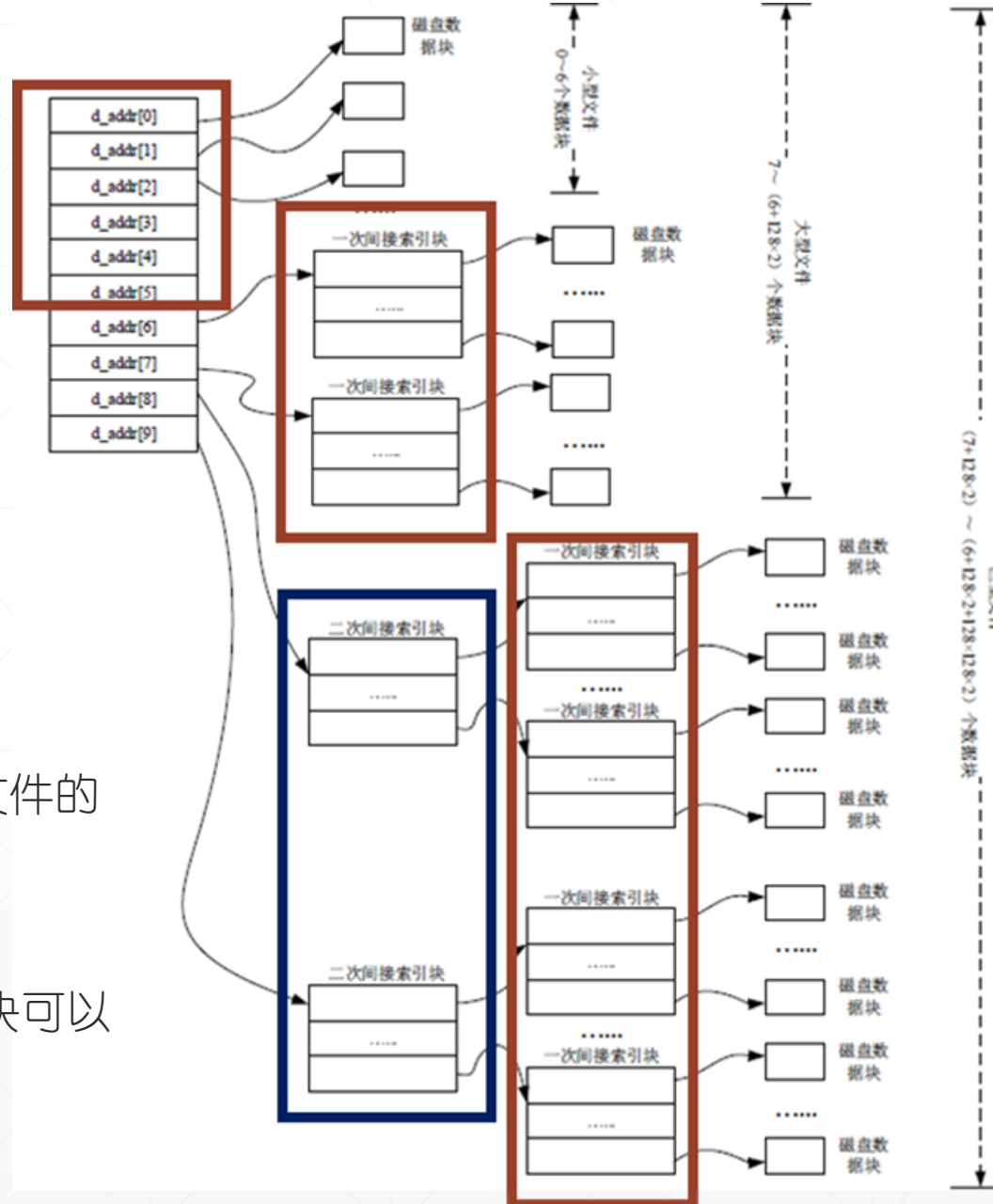
327

混合索引树

叶子节点是文件数据块
其它节点是索引块存放分配给文件的物理块号

DiskNode中的d_addr数组是混合索引树的根节点。

- d_addr[0] ~ d_addr[5], 可以登记6个物理块号, 存放文件的0#~5#逻辑块。
- d_addr[6], 第1个索引块, 可以登记 128 个物理块号。
- d_addr[7], 第2个索引盘块。。。
- d_addr[8]和d_addr[9]是2 次索引块。每个 2 次索引盘块可以挂 128 个一次索引盘块, 用来支持很大的文件。

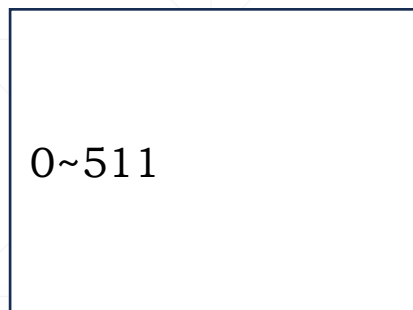


例：50#磁盘文件b：d_size == 1000字节

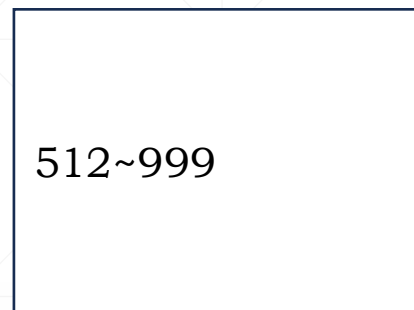
50#DiskInode

...	
d_mode	00
d_uid	12
d_size	1000
d_addr[0]	29
d_addr[1]	30
d_addr其余	0

29# 扇区



30# 扇区



b的目录项，存放在目录文件 John

50	"b\0"
----	-------



例：56#磁盘文件a： d_size == 4000字节

56#DiskInode

...	
d_mode	00
d_uid	12
d_size	4000
d_addr[0]	29
...	
d_addr[5]	34
d_addr[6]	35
d_addr其余	0

35# 扇区 (索引块)

36
37
其余0

29# 扇区

0~511

30# 扇区

512~1023

31# 扇区

1024...

32# 扇区

...

33# 扇区

~...

34# 扇区

...

36# 扇区

...

37# 扇区

...

a的目录项，存放在目录文件 John

56	"a\0"
----	-------

2、磁盘上的目录文件

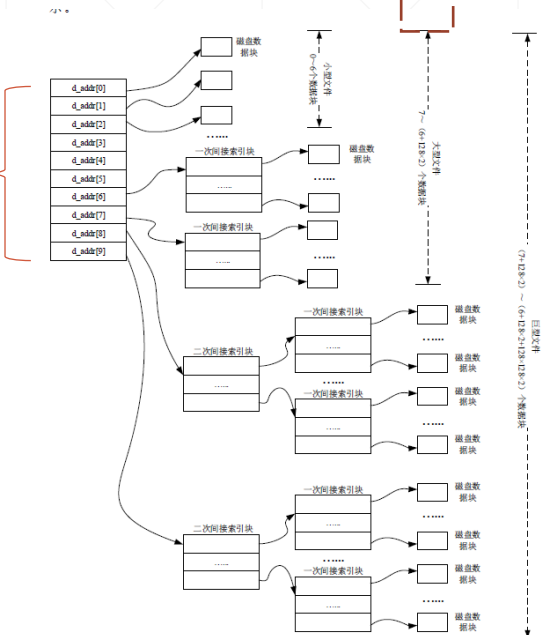
26#文件John

父目录home
的某个扇区 目录John的
某个扇区



1个DiskInode, 1个目录项,
1棵混合索引树 (树上挂着分
配给它的数据块和索引块)

```
class DiskInode
{
public:
    文件类型: 10
    unsigned int d_mode;
    int d_nlink;
    short d_uid;
    short d_gid;
    int d_size;
    int d_addr[10];
    int d_atime;
    int d_mtime;
};
```



```
class DirectoryEntry
{
public:
    static const int DIRSIZ = 28;
public:
    int m_ino; // 目录项中Inode编号
    char m_name[DIRSIZ]; // 目录项中路径名部分
};
```

John的目录项, 存放在目录文件 home

26	"John\0"
----	----------

例：目录文件John

26#DiskInode

...	
d_mode	10
d_uid	0
d_size	32*17
d_addr[0]	n
d_addr[1]	m
d_addr其余	0

n# 扇区

1	.
1	..
56	a
.....	
0	

m# 扇区

50	b
0	
0	
0...	
0	

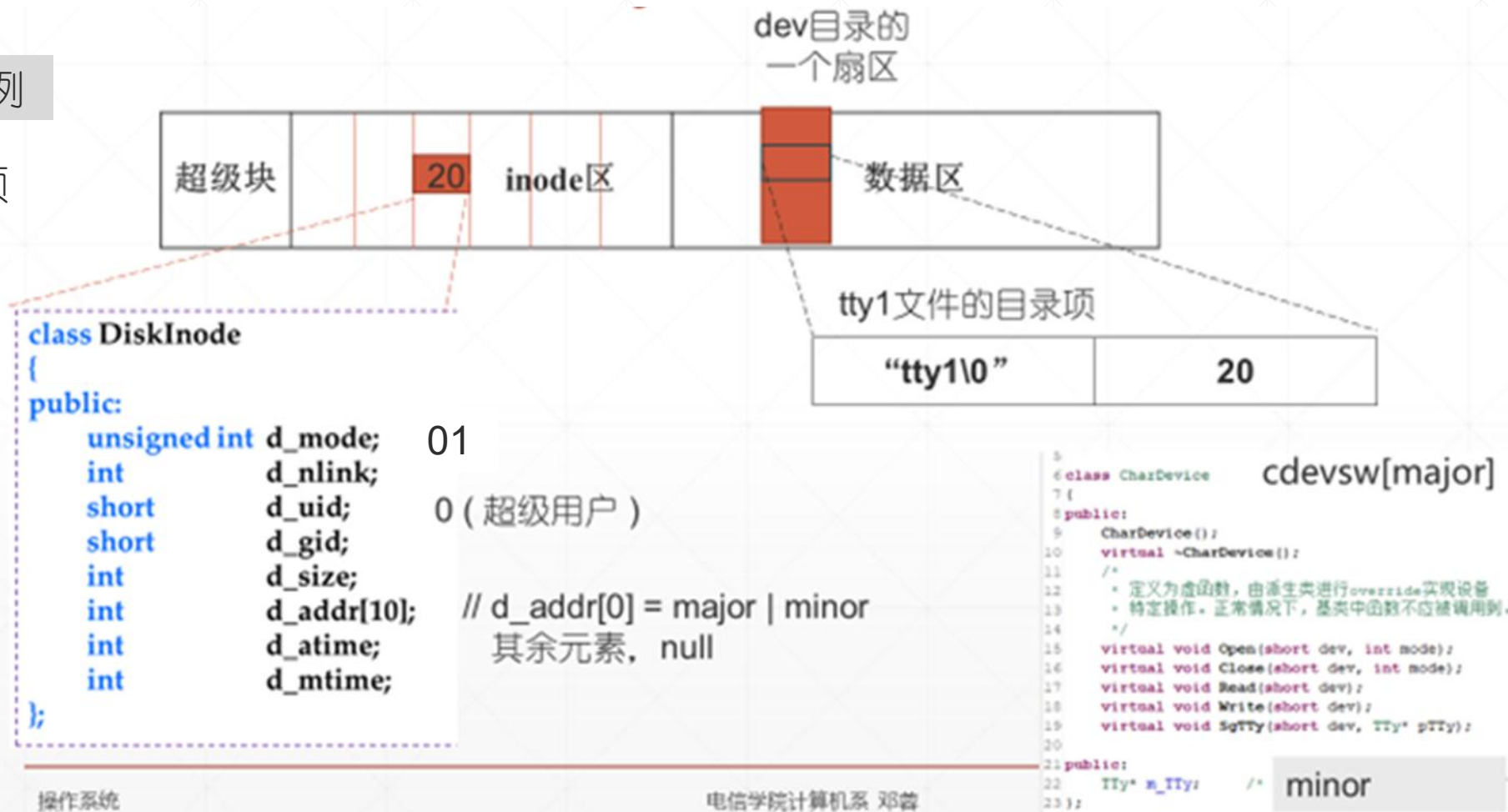
John的目录项，存放在目录文件 home

26	"John\0"
----	----------

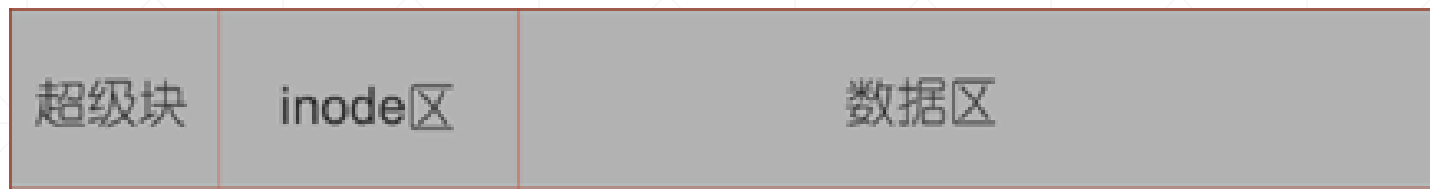
3、特殊类型的文件：字符设备文件和 块设备文件

字符设备文件 /dev/tty1为例

1个DiskInode, 1个目录项



八、磁盘空闲资源管理



数据区的存储资源是物理块
Unix V6, 1个扇区

空闲标识: 无
系统用空闲盘块号栈显式登记所有空闲数据块

2#扇区		3#扇区		4#扇区	
0# inode	8# inode				
1# inode	9# inode				
			■ ■ ■ ■ ■ ■		
7# inode	15# inode				

inode区的存储资源是DiskInode
空闲标识:
• d_mode中IALLOC为0
系统为了提高搜索效率, 设置空闲Inode栈,
管理100个空闲Inode号

DiskInode的编号, 由存储位置决定。n# DiskInode存放位置: 扇区号 $n/8 + 2$, 扇区内偏移量是 $(n\%8) * 64$

SuperBlock, 空闲盘块号栈 和 空闲Inode栈



```
11 class SuperBlock
12 {
```

```
21 public:
```

```
22     int     s_isize;        /* 外存inode区占用的盘块数 */
```

```
23     int     s_fsize;        /* 盘块总数 */
```

```
24
```

```
25     int     s_nfree;        /* 直接管理的空闲盘块数量 */
```

```
26     int     s_free[100];    /* 直接管理的空闲盘块索引表 */
```

```
27
```

```
28     int     s_ninode;       /* 直接管理的空闲外存inode数量 */
```

```
29     int     s_inode[100];   /* 直接管理的空闲外存inode索引表 */
```

```
30
```

```
31     int     s_flock;        /* 封锁空闲盘块索引表标志 */
```

```
32     int     s_iloc;        /* 封锁空闲inode表标志 */
```

```
33
```

```
34     int     s_fmod;        /* 内存中super block副本被修改标志, 意味着需要更新外存对应的Super Block */
```

```
35     int     s_ronly;        /* 本文件系统只能读出 */
```

```
36     int     s_time;        /* 最近一次更新时间 */
```

```
37     int     padding[47];    /* 填充使SuperBlock块大小等于1024字节, 占据2个扇区 */
```

```
38 };
```

} 空闲盘块号栈的首部。

s_free保存s_nfree个空闲盘块号。

} 空闲inode号栈。

s_inode数组保存s_ninode个空闲Inode号。

空闲inode的分配和回收

- 新建文件/目录，分配一个空闲inode: `num = ialloc (dev)`
- 删除文件/目录，回收一个空闲inode: `ifree (dev, num)`

s_ninode	
s_inode	789
	...
	253

s_ninode↑
 空闲
 DiskInode

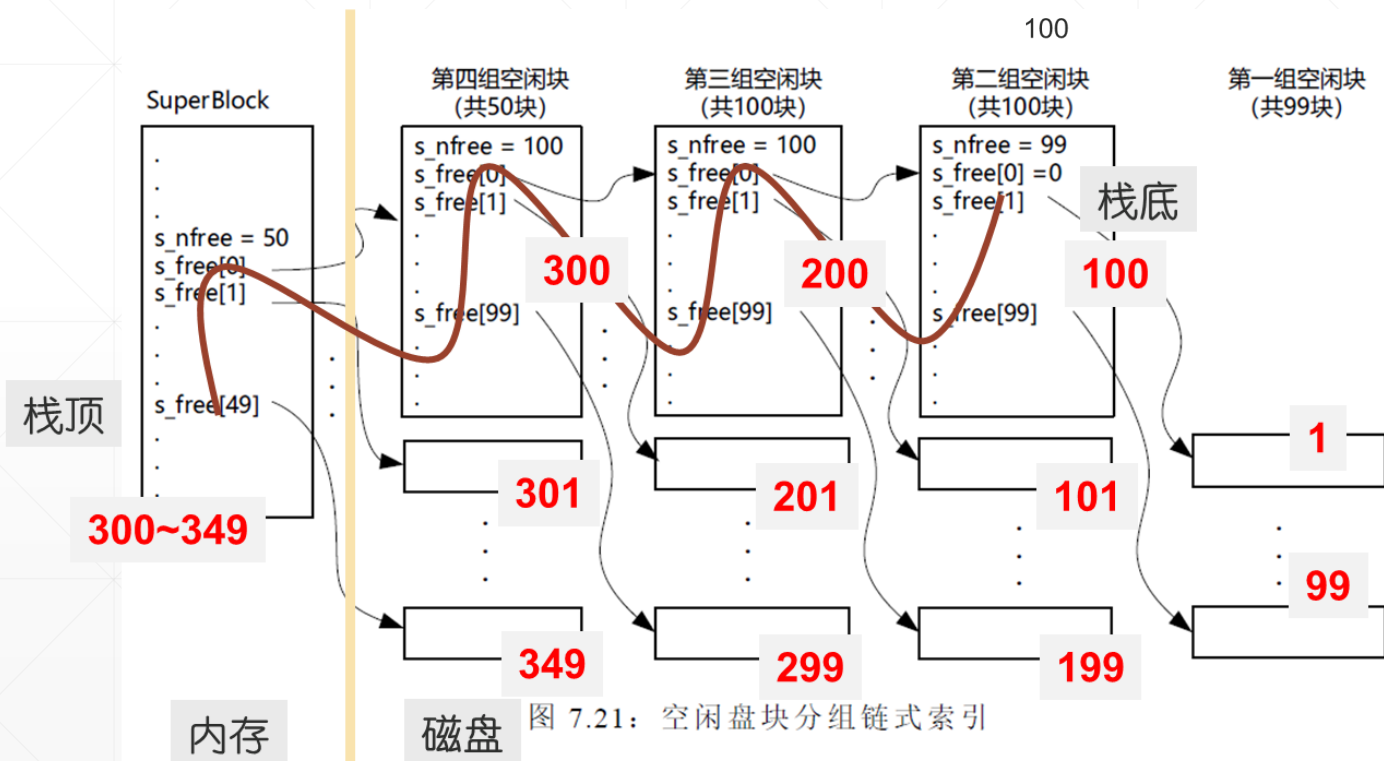
分配i节点: `ialloc( dev)`, 取空闲i节点栈的栈顶节点
 again: `if (s_ninode)`
 `return(s_inode[--s_ninode]);`
 else {
 `get next 100 free inodes from disk;`
 `goto again; }`

回收num#的i节点: `ifree(dev,num)`
`if (s_ninode != 100)`
 `s_inode[s_ninode++] = num;`

Unix V6, 从1#DiskInode开始, 逐个判断 inode 的使用状态。空闲DiskInode判断条件, PPT15。
 可以优化: 记住号码最小的空闲DiskInode。s_ninode为0时, 把它分配出去; 值低于阈值, 启动后台线程寻找空闲DiskInode。

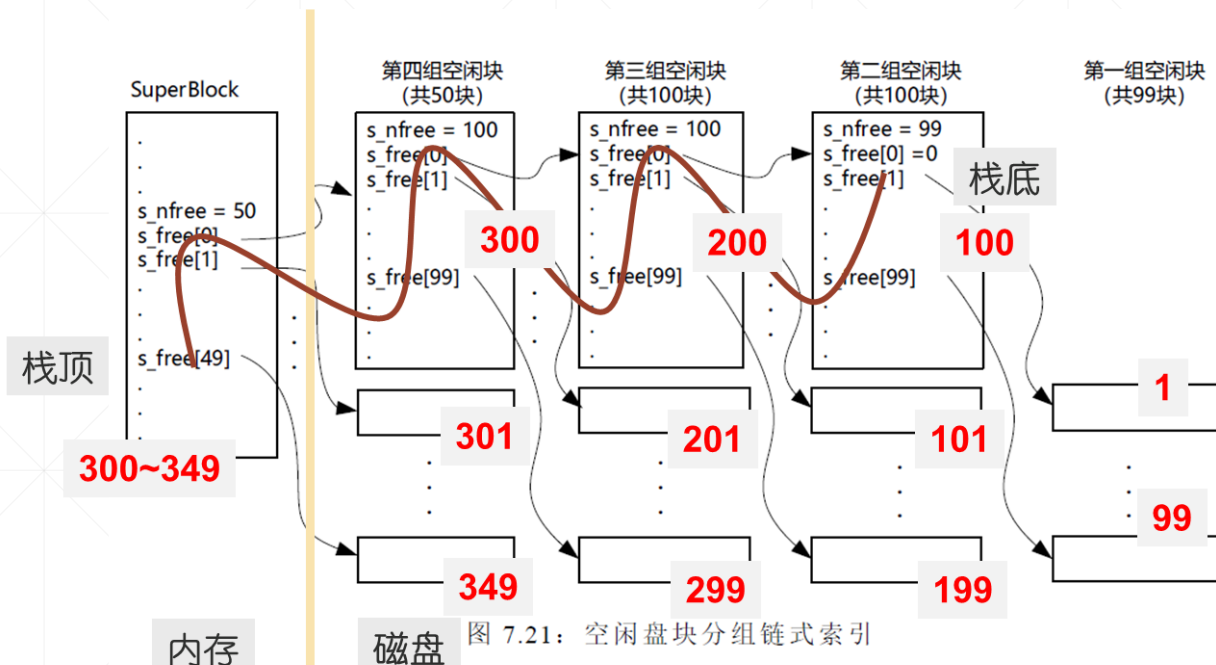
空闲盘块的分配和回收

- 成组链接法使用空闲盘块号栈管理所有空闲盘块。
- 栈，存放在超级块和空闲块中，显式登记所有空闲盘块号。
 - 第1组空闲块，99个
 - 第2组 ~, 100个
 - 最后1组 ≤ 100 个
 - 第1组的块号登记在第2组组长盘块
 - 第n组的块号登记在第n+1组组长盘块
 - 最后1组，登记在SuperBlock, s_nfree是空闲盘块数量



分配和回收操作

	s_nfree (50)
s_free	300
	
	349



为文件分配数据块num: num=FileSystem::Alloc()

- 1、n = s_free[--s_nfree]; // pop
- 2、if (s_nfree==0)
n#扇区开头的404字节, 复制到SuperBlock
覆盖s_nfree和s_free;
- 3、return(n); // n是分配出去的数据块号

文件释放数据块num: FileSystem::free(num)

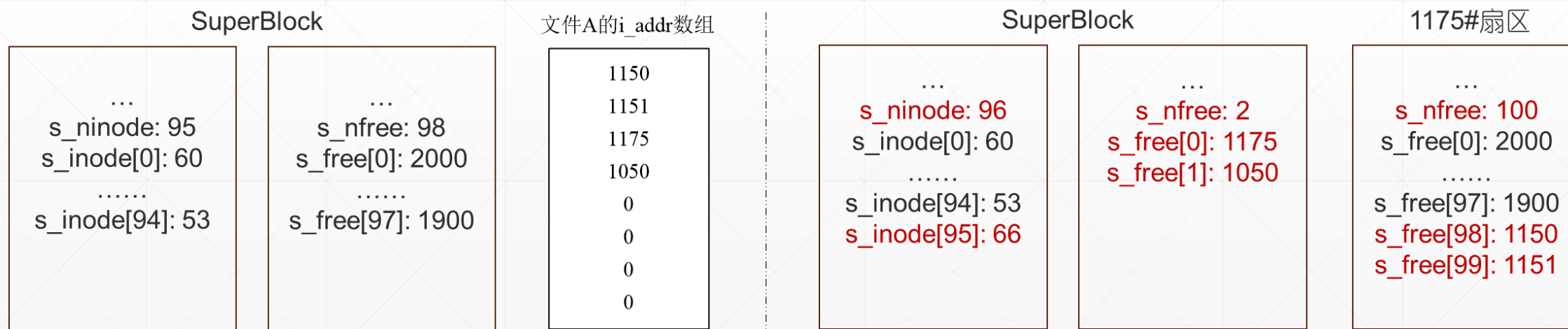
- 1、if (s_nfree==100) // num是新空闲块组长
把SuperBlock中的s_nfree和s_free复制到num#扇区;
s_nfree=0;
- 2、s_free[s_nfree++] = num; // push

例:

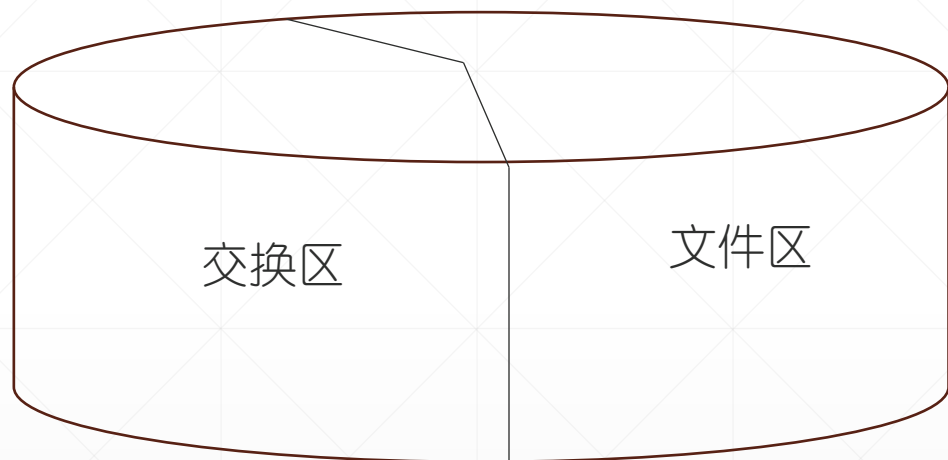
- 某Unix V6++文件卷，SuperBlock状态如下图1。66#文件A，混合索引表状态如下图2。请图示删除文件A后，SuperBlock的状态变化。

删除文件A (1) 66#DiskInode空闲、压入空闲inode栈
 (2) 分配给文件A的数据块1150, 1151, 1175, 1050空闲，压入空闲盘块号栈
 (3) SuperBlock有更新，置脏标记 $s_fmod = 1$ (未来，关机的时候，SuperBlock要写回磁盘)

图1



九、Unix V6++的主硬盘



组成

- 引导扇区，512字节的小程序，加载内核
- 内核 kernel程序
- 交换区（连续存储管理方式）
 - 进程图像占据连续存储单元
 - IO速度快。换入、换出操作追求速度。
- 基本文件系统（离散存储管理方式）
 - 文件可以使用不连续的物理块
 - 磁盘空间利用率高，没有碎片问题。
 - 单个文件，最后一块放不满，平均浪费半个数据块。

引导扇区

超级块200、201，inode区202~1023，其余数据区

c.img

0

kernel

[200, 18000)文件区

[18000,) 交换区

定义在 `FileSystem.h`

系统初始化时：加载主硬盘超级块，根目录，设置当前工作目录

```
/****** 1、加载超级块 *****/
```

```
Kernel::Instance().GetFileSystem().LoadSuperBlock(); // g_spb
```

```
/****** 2、加载根目录 *****/
```

```
FileManager& fileMgr = Kernel::Instance().GetFileManager();
```

```
fileMgr.rootDirInode = g_InodeTable.IGet(DeviceManager::ROOTDEV, FileSystem::ROOTINO); // 1
```

```
fileMgr.rootDirInode->i_flag &= (~Inode::ILOCK);
```

```
/****** 3、设置当前工作目录 *****/
```

```
User& us = Kernel::Instance().GetUser();
```

```
us.u_cdir = fileMgr.rootDirInode;
```

```
Utility::StringCopy("/", us.u_curdir);
```

系统正常运行期间

- (1) SuperBlock常驻内存，管理磁盘空闲资源
- (2) 根目录常驻内存，是绝对路径目录搜索的起点
- (3) 当前工作目录常驻内存，系统执行cd命令换1个，是相对路径目录搜索的起点